

FOCUSSENSE - DOCUMENTO TÉCNICO

Taller 1 - Desarrollo de una Aplicación Adaptativa | 2026-2

1. Descripción de la aplicación

FocusSense es un temporizador móvil de enfoque de 25 minutos que evita que el usuario tenga que reconfigurar la interfaz cuando cambia su entorno. La aplicación detecta condiciones reales del dispositivo y modifica automáticamente su presentación y consumo visual. El objetivo se alinea con el taller: responder dinámicamente al contexto mediante software adaptativo, procesamiento en tiempo real, capacidades del dispositivo y diseño modular [1].

2. Contexto utilizado

Se utilizan dos variables reales: (a) luminosidad ambiental en lux, obtenida mediante TYPE_LIGHT; y (b) porcentaje de batería y estado de carga, obtenidos mediante ACTION_BATTERY_CHANGED. Si un equipo no posee sensor de luz, la adaptación por batería permanece disponible y la interfaz informa el fallback.

3. Comportamiento adaptativo

Condición	Decisión	Adaptación automática
< 40 lux	Modo noche	Paleta oscura para disminuir deslumbramiento.
> 1000 lux	Modo exterior	Alto contraste para mejorar legibilidad.
Batería <= 20% y sin cargar	Modo ECO	Desactiva animaciones y oculta contenido secundario.
Condiciones combinadas	Modo combinado	Aplica noche/exterior y ECO simultáneamente.

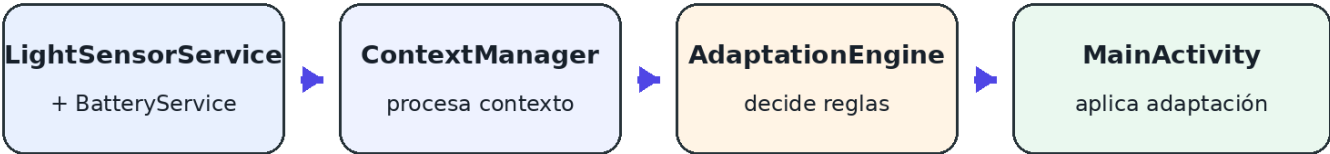
La lógica usa histéresis (umbrales distintos para entrar y salir de los modos de iluminación), evitando oscilaciones visuales cuando el sensor fluctúa cerca de un límite.

4. Pipeline adaptativo

CONTEXTO → PROCESAMIENTO → DECISIÓN → ADAPTACIÓN

Luminosidad + batería → integración y estabilización → reglas del motor → cambio de paleta, contraste, animaciones y contenido.

5. Arquitectura / componentes principales



6. Tecnologías utilizadas

Lenguaje: Kotlin.

Plataforma: Android nativo, API mínima 26 (Android 8.0), compile/target SDK 35.

APIs relevantes: SensorManager / Sensor.TYPE_LIGHT; BatteryManager; BroadcastReceiver; Handler; ObjectAnimator.

Construcción: Gradle 8.9, Android Gradle Plugin 8.7.3 y Kotlin Android Plugin 2.0.21.

Dependencias: No se requieren librerías de terceros en tiempo de ejecución.

7. Ubicación del código relevante

Elemento	Archivo / clase
Captura de luz	context/LightSensorService.kt
Captura de batería	context/BatteryService.kt
Procesamiento	context/ContextManager.kt
Decisión	adaptation/AdaptationEngine.kt
Modelo de contexto	model/ContextSnapshot.kt
Adaptación / interfaz	ui/MainActivity.kt

8. Metodología de desarrollo aplicada

Se adoptó Adaptive Software Development (ASD) porque el material teórico lo presenta como un enfoque iterativo, orientado a componentes, tolerante a cambios y guiado por aprendizaje. Su ciclo especular-colaborar-aprender [2] se aplicó así: (1) especular: misión, riesgos e iteraciones; (2) colaborar: separación por componentes con interfaces simples; y (3) aprender: revisión de calidad, funcionalidad y riesgos después de cada iteración.

Iteración	Objetivo	Resultado
1	Capturar contexto real	Servicios de luz y batería + ContextSnapshot.
2	Decidir y adaptar	Motor de reglas, histéresis y modos visuales.
3	Robustecer y demostrar	Fallback, prueba del motor, documentación y demo.

9. Validación y demostración

El archivo tools/EngineSmokeTest.kt valida seis escenarios: normal, noche, exterior, ECO, noche+ECO y exterior+ECO. En la sustentación, el comportamiento puede demostrarse cubriendo/iluminando el sensor y variando la batería desde los controles del emulador. La adaptación es automática y observable; no depende de un cambio manual dentro de la aplicación.

Referencias

[1] Universidad Nacional de Ingeniería. Taller 1: Desarrollo de una Aplicación Adaptativa, 2026-2.

[2] Maida, E. G. y Pacienza, J. Metodologías de desarrollo de software. Universidad Católica Argentina, 2015, sección 5.3.6 (Adaptive Software Development).